STARK-DNS: Offline Post-Quantum DNSSEC Resolution

Anonymous Authors
Affiliation withheld for double-blind submission

***Abstract*—The Domain Name System Security Extensions (DNSSEC) authenticate DNS responses through a hierarchical chain of RRSIGs over RSA-SHA256, ECDSA-P256, and Ed25519 signatures. Migrating these algorithms to NIST post-quantum signatures is operationally unresolved at Internet scale. ML-DSA-65 has not been introduced to DNSSEC: no IANA algorithm code point is assigned. The obstacle is wire-level: a single ML-DSA-65 signature is $3{,}293$ bytes and its public key is $1{,}952$ bytes, so the three RRSIGs alone in a typical chain (root-KSK $\rightarrow$ TLD-KSK $\rightarrow$ leaf) total $\approx 9{,}879$ bytes — approximately $8\times$ the $1{,}232$-byte UDP cap that DNS relies on before counting the accompanying DNSKEY records and DS hashes.**

We present *STARK-DNS*, a non-disruptive wrapper around DNSSEC. A trusted prover — or a Byzantine-fault-tolerant swarm of provers — executes DNSSEC validation *inside* a STIR-based STARK using in-circuit verifiers for the three deployed classical algorithms, commits the validated records to a Merkle tree, and signs the proof with ML-DSA-65. Edge consumers fetch the resulting epoch package once per epoch *out-of-band* (not through DNS/UDP), verify it once, and thereafter resolve queries offline against the committed RRsets. This is not a hybrid signature scheme: classical signatures are inputs to the prove pipeline, but the consumer's trust chain rests on post-quantum primitives only — ML-DSA-65, SHA-3, and STARK soundness. Authenticated denial-of-existence (NSEC/NSEC3 with wildcard synthesis) is out of current scope.

***STARK-DNS* performance. On AWS Graviton4 (`r8g.4xlarge` spot) at NIST PQ Level 3, the prover in-circuit verifies the three deployed DNSSEC algorithms in $0.68\,$s (RSA-2048), $105\,$s (Ed25519, $K{=}256$), and $95\,$s (ECDSA-P256) per record; edge verification (one STARK verify plus one ML-DSA-65 verify) completes in $\sim 1\,$ms per epoch and per-query offline resolution costs $\sim 1\,\mu$s. Single-record cost is $\sim \$0.002$ for ECDSA-P256. Assuming RSA-2048 leaves ($\sim 1\,$s/record sharded), a `.com`-scale (~ 160M DNSSEC-signed leaf RRsets) signing pass projects to ~ 43 hours on a $1{,}000$-worker swarm at $\sim\$3{,}010$ one-time cost; ECDSA/Ed25519 sharded projections are reported in the body. We give a machine-checked Tamarin proof of the swarm prover protocol.**

Keywords: DNS security, DNSSEC, post-quantum cryptography, STARKs, verifiable computation, offline resolution, critical infrastructure, ML-DSA, Merkle trees

I. Introduction

The Domain Name System (DNS) is the most widely deployed distributed database on the Internet, mapping human-readable domain names to IP addresses for virtually every networked interaction. The Domain Name System Security Extensions (DNSSEC) [3], [5], [4] extend DNS with a chain-of-trust rooted at the signed root zone, providing cryptographic authentication of DNS responses via RSA-SHA256 (alg. 8), ECDSA-P256 (alg. 13), and Ed25519 (alg. 15) signatures at each delegation level [38]. For government, military, and critical national infrastructure, DNSSEC is not optional: it is the primary mechanism by which DNS responses are authenticated, and its compromise would enable cache poisoning, man-in-the-middle attacks, and infrastructure redirection at scale.

The CRQC threat [36], [30] makes DNSSEC's three deployed algorithms migration targets (threat model in §III); the migration itself is *operationally unresolved at Internet scale*, and that is the problem this paper addresses. NIST has standardised three post-quantum signature schemes (ML-DSA / FIPS 204 [31], SLH-DSA / FIPS 205 [33], and the forthcoming FN-DSA / FIPS 206), but **ML-DSA-65 has not been introduced to DNSSEC**: no IANA algorithm code point has been assigned, and IETF DNSOP drafts [23], [35] on PQ-DNSSEC have not converged on a deployable solution. The obstacle is wire-level. An ML-DSA-65 signature is $3{,}293$ bytes and its public key is $1{,}952$ bytes. In a typical three-level DNSSEC chain (root-KSK $\rightarrow$ TLD-KSK $\rightarrow$ leaf-RRSIG), the three RRSIGs alone total $\approx 9{,}879$ bytes — approximately $8\times$ the $1{,}232$-byte UDP payload that EDNS0 [15] enables and DNS Flag Day operational guidance recommends to avoid IP fragmentation [16] — before counting the accompanying DNSKEY records (each $+1{,}952$ B public-key payload plus $+3{,}293$ B RRSIG) and DS hashes. Any in-band migration therefore requires fragmentation, TCP fallback, or protocol-breaking changes to the DNS wire format.

This paper takes a different approach. Rather than migrating the DNS infrastructure to post-quantum cryptography in-band — a process that requires coordinated changes across authoritative nameservers, recursive resolvers, registrars, and stub resolvers worldwide — we *wrap* the existing infrastructure in a post-quantum verification layer that can be deployed today, requires no changes to any existing DNS component, and provides offline resolution capability as a bonus. We emphasise upfront that this is *not* a hybrid signature scheme: classical DNSSEC signatures are inputs to our prove pipeline, but the consumer's trust chain rests on post-quantum primitives only (§V). Because the post-quantum artefact is distributed out-of-band (not through DNS/UDP), the $1{,}232$-byte wire-size constraint and the missing IANA code point are sidestepped.

Network and Distributed System Security (NDSS) Symposium 2027
XX – XX
ISBN XXX-X-XXXXXXXXX-X
https://dx.doi.org/XX.XXXXX/ndss.2027.XXXXX
www.ndss-symposium.org

A. Core Idea

A trusted prover periodically performs full DNSSEC resolution for a configured set of domains, capturing and verifying the complete chain of trust from the root Key Signing Key (KSK) through each delegation to the leaf Resource Record Signatures (RRSIGs). The prover then proves the correctness of this entire verification inside a Scalable Transparent Argument of Knowledge (STARK) [10] circuit, operating over FIPS-aligned parameters [7], adapted with STIR [6] for low-degree testing. The verified records are committed to a Merkle tree whose root hash is included as a public input to the STARK proof, cryptographically binding the data structure to the proof of its provenance. The prover signs the STARK proof with an ML-DSA post-quantum digital signature, binding the proof to an authenticated identity.

The resulting *epoch package* is distributed to edge devices, which verify it once (one STARK verification, one ML-DSA signature verification) and then resolve DNS queries entirely offline via Merkle inclusion proofs against the trusted root. Each query requires approximately 20 hash computations for a tree of one million records—no signatures, no STARK re-verification, no network access.

Per-record proofs are bound to their (record, RRSIG) witnesses, not to an epoch, so the prover reuses them across epochs while RRSIGs are valid and unrevoked; each new epoch proves only the delta (§VII-D).

B. Motivating Deployment Environments

Two orthogonal operational requirements converge on the same technical solution. *Post-quantum protection for critical infrastructure DNS without waiting for ecosystem-wide PQC migration*: `.gov`, `.mil`, and national critical-infrastructure zones are high-value targets whose DNSSEC records are protected only by RSA/ECDSA, vulnerable to silent on-path Shor-forgery as detailed in §III-A; the IETF PQ-DNSSEC migration path has no clear timeline. *Offline DNS resolution with cryptographic guarantees*: military systems in contested or communications-denied environments, air-gapped networks, industrial control systems, and bandwidth-constrained edge devices need authenticated DNS without reliable connectivity, which existing offline solutions (caching, AXFR/IXFR) cannot provide. Our architecture addresses both with one wrapper.

C. Contributions

- **Architecture and construction.** STARK-DNS, a non-disruptive wrapper around existing DNSSEC, defining a *succinct, non-interactive, post-quantum authenticated data structure with provenance* that binds prover identity, data integrity, data provenance, temporal validity, and efficient lookup into a single cryptographic object (§IV).
- **Security analysis.** A formal analysis under a quantum threat model, proving four properties (record integrity, prover authentication, non-substitutability, epoch freshness) and quantifying the residual attack surface (§V).

TABLE I
SIGNATURE AND PUBLIC KEY SIZES FOR DNSSEC-RELEVANT ALGORITHMS. PQC SIZES FROM NIST FIPS 204/205.

Algorithm	Signature (B)	Public Key (B)
ECDSA P-256	64	64
Ed25519	64	32
RSA-2048	256	256
ML-DSA-44	2,420	1,312
ML-DSA-65	3,293	1,952
ML-DSA-87	4,627	2,592
SLH-DSA-128s	7,856	32
SLH-DSA-256s	29,792	64

- **Implementation and benchmarks.** An end-to-end prototype with measured proof generation, STARK verification, per-query Merkle verification, and storage overhead at realistic zone sizes (§VI).
- **Practical incremental operation.** Per-record STARK proofs are bound to (record, RRSIG) and reused across epochs until DNSSEC signatures expire or the record is revoked, so steady-state per-epoch prover work is the delta only (§VII-D).

II. BACKGROUND

A. DNSSEC Architecture

DNSSEC [3], [5], [4] authenticates DNS responses through a hierarchical chain of trust anchored at the signed root zone. Each zone publishes DNSKEY records and signs its resource record sets (RRsets) with RRSIG records; delegation is authenticated via DS records in the parent zone, which hash the child's Key Signing Key (KSK). A validating resolver follows DS→DNSKEY delegations from the root trust anchor down to the target zone's RRSIG over the queried record—a minimum of three signature verifications for a typical second-level domain. The deployed signature algorithms are RSA-SHA256 (alg. 8), ECDSA P-256 (alg. 13), and Ed25519 (alg. 15) [38]; all three are vulnerable to Shor's algorithm.

B. The PQC Migration Problem for DNS

The NIST post-quantum signature standards produce signatures and public keys dramatically larger than their classical counterparts. Table I summarises the relevant sizes.

A typical DNSSEC response with full chain fits within a single 1,232-byte UDP payload (the EDNS0 recommended size [15]) under ECDSA. With ML-DSA-65 the three-level chain's signatures alone total $\approx 9{,}879$ bytes, requiring fragmentation, TCP fallback, or protocol redesign. IETF DNSOP has explored hybrid signatures, hash-chain compression, and zone-transfer distribution [23]; none has consensus and all require coordinated changes across authoritative servers, resolvers, and in some cases registrars.

C. STARKs and Verifiable Computation

A Scalable Transparent Argument of Knowledge (STARK) [10] allows a prover to convince a verifier

that a computation was performed correctly, with three properties relevant to our construction.

First, *succinctness*: the proof size and verification time are polylogarithmic in the size of the computation, so a proof that millions of cryptographic operations were performed correctly can be verified in milliseconds.

Second, *transparency*: the proof system requires no trusted setup ceremony and no secret parameters—the only cryptographic assumption is the collision resistance of the hash function used for Merkle commitments and the Fiat–Shamir transcript.

Third, *post-quantum security*: because STARK soundness rests on hash-function collision resistance rather than number-theoretic assumptions, STARKs are not vulnerable to Shor's algorithm. Concrete post-quantum security under the quantum random oracle model (QROM) has been established for FIPS-aligned STARK constructions using SHA-3 commitments [7].

A STARK proof has *public inputs*—values that both the prover and verifier see—and a *witness*—the private data that the prover uses to generate the proof but does not reveal. The proof convinces the verifier that the prover knows a witness satisfying a specified computation with respect to the public inputs, without revealing the witness itself.

D. Merkle Trees and ML-DSA

A Merkle tree [26] commits to a dataset via a binary hash tree; an inclusion proof for a leaf is its $\lceil \log_2 N \rceil$ sibling hashes ($\sim$20 hashes for $N = 10^6$). ML-DSA [31] is the NIST post-quantum signature standard built on Module-LWE; ML-DSA-65 (NIST Level 3) produces 3,293-byte signatures and verifies in <0.5 ms on modern hardware.

III. Threat Model

A. Adversary Capabilities

We consider an adversary $\mathcal{A}$ with the following capabilities.

Quantum computation. $\mathcal{A}$ has access to a cryptographically relevant quantum computer capable of running Shor's algorithm. This means $\mathcal{A}$ can break RSA, ECDSA, Ed25519, and all number-theoretic signature schemes in polynomial time. In particular, $\mathcal{A}$ can forge any existing DNSSEC RRSIG signature.

Network position. $\mathcal{A}$ can observe, inject, modify, drop, or replay packets on any network path—between recursive resolvers and authoritative nameservers, between the prover and edge devices, and over any channel carrying epoch packages.

Silent quantum-MITM baseline. Combining the two capabilities yields a baseline attack on DNSSEC-as-deployed: $\mathcal{A}$ recovers a zone's ZSK via Shor, intercepts a victim's query, and returns a forged RRset signed under the recovered ZSK with all upstream elements genuine. The response passes every classical validator with no protocol-level artefact—silent and undetectable, repeatable until the next ZSK rotation. STARK-DNS removes this surface from the consumer side (§V-C).

Quantum oracle access. $\mathcal{A}$ may make up to q quantum queries to the hash function modelled as a random oracle. We consider $q \in \{2^{40}, 2^{65}, 2^{90}\}$ following the NIST post-quantum security framework [32].

B. Adversary Limitations

Hash function security. $\mathcal{A}$ cannot break collision or preimage resistance of SHA-3 [29]. Best known quantum speedups: $\approx 2^{n/2}$ for preimages (Grover [19]), $\approx 2^{n/3}$ for collisions (BHT). We use SHA3-512 at Level 3 ($\approx 2^{170}$ collision, $\geq 2^{256}$ preimage); SHA3-256 suits lower targets ($\approx 2^{85}$ collision). Concrete QROM bounds: the compressed-oracle analysis of [39].

Lattice-based signature security. $\mathcal{A}$ cannot forge ML-DSA signatures under the Module-LWE assumption at the configured security level.

STARK soundness. $\mathcal{A}$ cannot produce a valid STARK proof for a false statement except with negligible probability bounded by the STARK's soundness error [7], [6].

Prover integrity. The trusted prover's ML-DSA private key is not compromised through non-cryptographic means. This is a standard operational security assumption analogous to trusting a Certificate Authority's key storage.

C. Trust Assumptions

(T1) Honest prover. The STARK prover is operationally secure and faithfully executes the proof-generation protocol.

(T2) DNSSEC chain integrity at proof time. The DNSSEC chain of trust is intact at the time the prover captures and verifies it. We do not assume it remains intact after that point.

(T3) Hash collision resistance. The hash function used for STARK commitments, Merkle tree construction, and Fiat–Shamir compilation is collision-resistant against quantum adversaries at the configured security level.

D. Residual Attack: Pre-Proof Chain Compromise

The principal residual attack is compromise of the DNSSEC chain *before* the prover captures it. If $\mathcal{A}$ can forge DNSSEC signatures and inject fraudulent records into the chain during the window in which the prover is performing resolution, the prover will faithfully prove incorrect data: the resulting STARK is cryptographically sound (it correctly attests to whatever the prover *saw*), but the data it attests to is adversarial.

a) The attack reduces to a single forged leaf RRSIG.: A common but *incorrect* intuition holds that $\mathcal{A}$ must simultaneously forge signatures across multiple administrative levels (root, TLD, domain) to fool the prover. This is wrong. Under a quantum-on-path threat model, the attack is structurally cheaper: $\mathcal{A}$ recovers the target zone's ZSK private key from the (publicly published) ZSK public key via Shor's algorithm, then forges *one* RRSIG over a fraudulent leaf RRset that verifies under the unmodified, legitimately-published ZSK. The DNSKEY chain (KSK $\rightarrow$ ZSK), DS records, and parent-zone signatures all remain genuine, because the attacker leaves them untouched. The prover, executing standard DNSSEC chain validation, sees an authentic chain rooted at the trust anchor and a valid RRSIG over the leaf RRset, and accepts the leaf as correct.

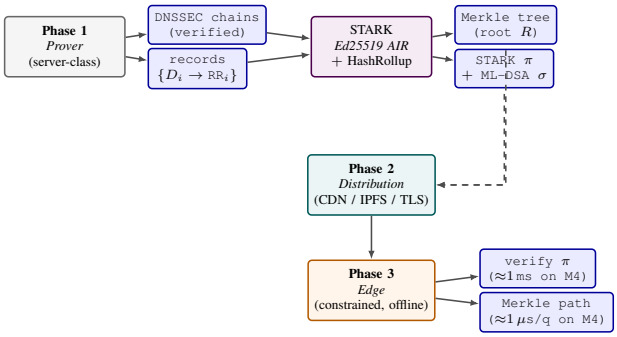

Fig. 1. Three-phase STARK-DNS pipeline. Phase 1 (*Prover*) captures DNSSEC chains and produces a STARK-proven, ML-DSA-signed epoch package. Phase 2 (*Distribution*) ships the package through any unauthenticated channel. Phase 3 (*Edge*) verifies once (one STARK proof, one ML-DSA signature) and serves unlimited offline DNS lookups via Merkle path verification.

The cost of the residual attack against a single zone is therefore *one Shor execution per ZSK* plus *on-path injection* of the forged leaf RRset to the prover's recursive resolver during the epoch window. There is no "multiplicative cost across ℓ administrative boundaries"; an attacker who compromises only the leaf zone's ZSK obtains the equivalent capability of forging records in that zone for as long as the ZSK rotation interval allows.

b) Mitigations.: Four design-level mitigations apply (analysed in §V-C): (M1) *multi-vantage resolution* via V BGP-disjoint recursive resolvers with byte-identical RRSIG agreement; (M2) *temporal jitter* forcing sustained rather than single-shot on-path presence; (M3) *cross-epoch hash-chaining* so retrospective contradictions trigger auditor-driven revocation; (M4) *tightened ZSK rotation*, costing one Shor execution per rotation rather than per ZSK lifetime. The prototype assumes a benign path; a V-of-V multi-vantage extension is follow-on work (§VII).

c) Security envelope (explicit).: Without multi-vantage resolution, STARK-DNS guarantees post-quantum integrity against consumer-side adversaries unconditionally (Theorem 1); it does *not* prevent quantum on-path forgery during the prover's epoch-capture window. With V-of-V multi-vantage (M1), the residual attack additionally requires the adversary to control all V BGP-disjoint paths during the same epoch — a conditional guarantee, as stated in §V-C.

IV. Architecture

The STARK-DNS architecture operates in three phases: proof generation at the prover (§IV-A), epoch package distribution (§IV-B), and offline resolution at the edge (§IV-D). Figure 1 shows the three-phase pipeline end-to-end. We then formalise the cryptographic binding structure (§IV-E).

A. Phase 1: Proof Generation

The prover is a server-class machine with network access to the DNS infrastructure. At the beginning of each epoch (a configurable interval; see §VII-B), the prover executes the following steps.

Step 1 (capture). For each domain D_i in the configured set $\mathcal{D} = \{D_1, \ldots, D_N\}$, the prover performs full recursive DNSSEC resolution and stores the wire-format records and signatures across the complete chain (root DNSKEY+RRSIG, each DS record + parent's RRSIG over the DS RRset, each intermediate DNSKEY+RRSIG, leaf RRset+RRSIG).

Step 2 (STARK proof). The prover encodes the chain verification as an AIR. The circuit verifies, per domain: (a) every signature in $\mathcal{S}_{\text{ic}}$ (Ed25519 in the prototype; ECDSA-P256 chain in-circuit at $K{=}256$ end-to-end with the residual 0.1% (Z^{-1}, $R.x \bmod n$, equality) verified natively at prove time, see Section VI-F); (b) each DS digest using the in-circuit gadget for its declared type (SHA-256 / DS type 2); (c) temporal validity of each in-circuit signature against the epoch timestamp; and (d) Merkle-tree construction over the verified records. Links in $\mathcal{S}_{\text{att}}$ (RSA at parent/root in today's DNS) are verified natively at capture and attested under T2, not re-verified inside the AIR (Definition 1). Public inputs are $(\mathsf{pk}, R, T, \mathsf{seq}, \mathsf{prev})$; the witness is the raw chain plus the intermediate values of in-circuit signature verification and the Merkle construction. Two prover paths exist: the **in-circuit verified** path proves every link in $\mathcal{S}_{\text{ic}}$ inside the AIR (sound under STARK soundness + SHA-3 collision resistance, independent of prover software, but scoped to $\mathcal{S}_{\text{ic}}$ per Theorem 1); the **attested-rollup** path stops at a STARK over the Merkle commitment of records the prover verified natively, relying on T1/T2 for per-signature validity. We label every benchmark and theorem "[in-circuit]" or "[attested rollup]" to keep the two security profiles distinct. The STARK uses the FIPS-aligned parameter framework of [7] with STIR [6] substituted as the low-degree test: SHA-3-only verifier-path commitments, concrete QROM security at the target NIST PQ level.

Step 3 (sign): $\sigma = \mathsf{Sign}(\mathsf{sk}, \pi \| R \| T \| \mathsf{seq} \| \mathsf{prev})$.

Step 4 (assemble): $\mathcal{E} = (\pi, \sigma, \mathsf{pk}, R, T, \mathsf{seq}, \mathsf{prev}, \mathcal{M})$, where $\mathcal{M}$ is the full or device-specific Merkle subtree. Per-record proofs are reused across epochs while their RRSIGs are valid and unrevoked, so steady-state prover work is the delta only (§VII-D).

B. Phase 2: Distribution

$\mathcal{E}$ is distributed via any available channel (push: pub/sub, broadcast, physical media; pull: HTTPS, FTP, satellite). The channel need not be authenticated or confidential: $\mathcal{E}$ is self-authenticating via the ML-DSA signature plus STARK proof, an interceptor learns only public DNS records, and tampering invalidates σ. Bandwidth-constrained devices receive a per-device package containing only the relevant $O(n \log N)$ Merkle subtree plus the shared (π, σ).

C. Provisioning and Key Rotation

Edge devices acquire the authority's ML-DSA-65 public key pk at provisioning time via an out-of-band channel (manufacturing pinning, signed firmware update, or operator-side configuration). Rotations are announced *inside* a trusted epoch package: each epoch carries a signed binding $(\mathsf{pk}_{\text{next}}, T_{\text{rotate}}, \mathsf{prev}_{\text{rotate}})$ committing the next key, rotation

time, and previous-epoch chain; edges accept $\mathsf{pk}_{\text{next}}$ only when delivered inside an authentic epoch under the current pk, preventing standalone-key substitution. Swarm-mode witness allowlists rotate by the same mechanism. The emergency key is generated in an air-gapped HSM at zone setup, never online; on compromise, the authority publishes a revocation epoch under this offline key, collapsing freshness across all edges within at most one epoch period.

a) Multi-epoch miss recovery and policy.: Devices behind by k epochs catch up via a sparse $O(\log_2 k)$ skip-list of past epoch roots signed under the current pk, preserving the strict-prev-chain property. Default freshness-violation policy is fail-open to standard DNSSEC; critical-infrastructure deployments may configure fail-closed (SERVFAIL on expired epoch or denial-of-existence queries outside the committed corpus).

D. Phase 3: Offline Resolution

Upon receiving an epoch package $\mathcal{E}$, the edge device performs *one-time epoch verification*:

Step 1 (ML-DSA verify, $<0.5\,ms$): verify σ over $\pi\|R\|T\|\mathsf{seq}\|\mathsf{prev}$ using pk. *Step 2 (STARK verify, 0.5–$5\,ms$ at NIST PQ Level 3):* verify π against public inputs $(\mathsf{pk}, R, T, \mathsf{seq}, \mathsf{prev})$. *Step 3 (freshness check):* require $\mathsf{seq} > \mathsf{seq}_{\text{prev}}$, T within the epoch window, and prev matching the device's last accepted package (or genesis on first install); cache R as the trusted root.

Thereafter, each DNS query is resolved offline:

a) Step 4: Per-Query Resolution.: For a query Q (e.g., "A record for `mail.example.gov`"), the device retrieves the corresponding leaf from the local Merkle tree and verifies the Merkle inclusion proof against R. If the proof verifies, the record is authentic. Cost: ~ 20 hash computations for a tree of 10^6 records, i.e., $\approx$1–2 μs on an Apple M4 ($\approx$8 μs on a Raspberry Pi 4).

b) Authenticated non-membership (scoped).: With leaves sorted lexicographically by $(\mathsf{name}, \mathsf{type})$, the two leaves bracketing a queried key prove non-membership for that exact key in the committed dictionary. This is *not* a drop-in replacement for full DNSSEC denial-of-existence semantics (wildcard synthesis, NXDOMAIN/NODATA, NSEC3 opt-out): faithful in-circuit NSEC/NSEC3 is identified as future work (§VII-F). The default policy on a denial-of-existence query is *fail-open* to standard DNSSEC validation; deployments where contestability of absence matters (military / critical-infrastructure) can configure the edge to *fail-closed*, returning SERVFAIL until in-circuit NSEC/NSEC3 lands.

c) CNAME chains and multi-step resolution.: The committed dictionary stores per-name RRsets keyed by canonical $(\mathsf{name}, \mathsf{type})$; a CNAME-redirected lookup performs successive offline lookups along the chain, each a Merkle inclusion proof against the same trusted root R. Chain truncation, redirection, and loops are all prevented by the per-step Merkle binding plus a configurable depth cap (default 8). Cross-zone CNAMEs landing outside the committed corpus terminate offline resolution and fail open to standard DNSSEC.

E. Cryptographic Binding Structure

The security of the construction rests on the fact that every component of the epoch package is cryptographically bound to every other component. We formalise this as follows.

Definition 1 (Epoch Package Binding)**.** *Let H denote SHA3-512 at Level 3 (SHA3-256 may be used at lower targets with the corresponding reduction in collision cost, §III-B), $\mathcal{S}_{\text{ic}}$ the set of DNSSEC signature/digest algorithms verified in-circuit by the AIR (Tab. II), and $\mathcal{S}_{\text{att}}$ its complement (verified natively under T2). Let $\mathsf{prev} = H(\pi_{\text{prev}} \,\|\, R_{\text{prev}} \,\|\, \mathsf{seq}_{\text{prev}} \,\|\, T_{\text{prev}})$ (or a genesis constant for the first epoch). An epoch package $\mathcal{E} = (\pi, \sigma, \mathsf{pk}, R, T, \mathsf{seq}, \mathsf{prev}, \mathcal{M})$ is* well-bound *iff: (i) σ is a valid ML-DSA signature under pk over $(\pi, \mathsf{pk}, R, T, \mathsf{seq}, \mathsf{prev})$; (ii) π is a valid STARK proof with public inputs $(\mathsf{pk}, R, T, \mathsf{seq}, \mathsf{prev})$; (iii) every node in $\mathcal{M}$ is committed under R, the Merkle root of the canonical lookup tree; (iv) for every leaf in $\mathcal{M}$, chain links in $\mathcal{S}_{\text{ic}}$ are verified inside the AIR and links in $\mathcal{S}_{\text{att}}$ are verified natively and rolled into π as attested inputs. Including prev realises the hash-chained epoch log (M3): any inter-epoch contradiction is publicly detectable.*

σ binds identity to proof, π binds provenance to data, R in public inputs binds the data structure to the proof, and T, seq bind time — every component cryptographically commits to every other.

F. The STARK Circuit for DNSSEC Verification

The prover trace lives over the Goldilocks field $\mathbb{F}_p$ ($p = 2^{64} - 2^{32} + 1$); verifier-path commitments use the sextic extension $\mathbb{F}_{p^6}$ following the FIPS-aligned framework of [7] with STIR [6] for low-degree testing. Design-wise the AIR encodes (i) signature verification for all three deployed DNSSEC algorithms—RSA (modular exponentiation $s^e \bmod n$), ECDSA (scalar multiplication plus field arithmetic), and Ed25519 (prime-field arithmetic over $\mathbb{F}_{2^{255}-19}$ with cofactored verify); (ii) DS record digest checks (digest types 1/2/4 = SHA-1/SHA-256/SHA-384); (iii) chain-of-trust traversal linking each RRSIG to its DNSKEY via parent-zone DS records up to the root; (iv) temporal validity comparing each RRSIG's `inception`/`expiration` against T; and (v) Merkle tree construction over the verified leaves. The *current prototype* implements the Ed25519 signature gadget and the SHA-256 (DS type 2) digest gadget in-circuit end-to-end; the ECDSA-P256 path has all in-circuit AIR primitives (F_p/F_n arithmetic, Renes–Costello–Batina 2016 complete addition, scalar-mult step, Fermat-style square-and-multiply) implemented and validated to full K=256 end-to-end with the chain in-circuit and the final 0.1% (group-add, Z^{-1}, $R.x \bmod n$, equality) verified natively at prove time (Section VI-F); RSA-2048 leaves are in-circuit (e=65,537 chain), with parent/root RSA and DS types 1/4 in $\mathcal{S}_{\text{att}}$ (Tab. II). Per-record cost is dominated by signature verification ($\sim 4\times10^7$ constraints for ECDSA-P256, $\sim 7.7\times10^7$ for Ed25519 at K=256); total circuit size is linear in N with average chain depth ≈ 3.

a) Algorithm and digest agility.: DS digest types 1 (SHA-1, deprecated by RFC 4509) and 4 (SHA-384) are pending in-circuit gadgets; a DS-384 gadget costs $\approx 1.6\times$ a SHA-256 gadget per compression call. Signature algorithms beyond those in Tab. II enter $\mathcal{S}_{\mathrm{ic}}$ as their AIRs are added. Name canonicalisation (RFC 4034 §6.2–6.3) and label-decompression are applied in the witness pre-processor; TTLs are bound at proof time via T, with an optional edge-side policy restricting serving lifetime to $\min(TTL, \Delta T)$.

V. Security Analysis

We prove four security properties and analyse the residual attack surface. Throughout this section the algorithm-coverage split of Definition 1 applies: Property 1 is restricted to chain links whose algorithm lies in $\mathcal{S}_{\mathrm{ic}}$ (links in $\mathcal{S}_{\mathrm{att}}$ are protected by T2 native-verification rather than by an in-circuit guarantee), while Properties 2–4 are algorithm-coverage–independent: they bind prover identity, the proven-data-to-proof commitment, and epoch freshness, and hold for every well-bound epoch package regardless of which chain links were proved versus attested.

A. Property 1: Post-Quantum Record Integrity

Theorem 1 (Post-Quantum Record Integrity)**.** *Restricted to chain links whose signature algorithm lies in $\mathcal{S}_{\mathrm{ic}}$ (Definition 1), no quantum adversary $\mathcal{A}$ can cause an edge device to accept a DNS record that was not DNSSEC-valid at proof generation time T, assuming: (i) collision resistance of H against quantum adversaries at level λ, with adversarial advantage ε_H bounded by the QROM security of SHA-3 (e.g., $\varepsilon_H \leq \mathrm{negl}(\lambda)$; see [30] for compressed-oracle bounds); (ii) soundness of the STARK proof system with error ε_π; and (iii) EUF-CMA security of ML-DSA at security level λ. Links whose algorithm lies in $\mathcal{S}_{\mathrm{att}}$ inherit the integrity of native prover verification under T2 and are not covered by this in-circuit guarantee.*

Proof. Suppose $\mathcal{A}$ produces a record r^* that was not in the original DNSSEC-verified set but is accepted by the edge verifier. The edge verifier checks a Merkle inclusion proof for r^* against the trusted root R. For r^* to be accepted, one of three events must occur:

Event E_1 (Merkle forgery): $\mathcal{A}$ produces a valid Merkle inclusion proof for r^* against R without r^* being a leaf of the committed tree. This requires a collision or second-preimage in H; the corresponding QROM bound is negligible at our chosen parameters (λ=192 with SHA3-512, sponge capacity c=512), with concrete q-versus-c tradeoffs given by the Zhandry compressed-oracle analysis [39].

Event E_2 (STARK forgery): $\mathcal{A}$ produces a valid STARK proof π^* with a fraudulent Merkle root R^* as public input, where R^* is the root of a tree containing r^*. This violates STARK soundness, occurring with probability at most $\varepsilon_\pi \leq \varepsilon_{\mathrm{IT}} + \varepsilon_{\mathrm{bind}} + \varepsilon_{\mathrm{FS}}$ by the modular three-term QROM decomposition of [7], with $\varepsilon_{\mathrm{IT}}$ inherited from the STIR proximity-test bound [6].

Event E_3 (Signature forgery): $\mathcal{A}$ produces a valid ML-DSA signature σ^* on a proof containing R^* without possessing sk.

TABLE II
Current $\mathcal{S}_{\mathrm{ic}}$ vs $\mathcal{S}_{\mathrm{att}}$ coverage. Ed25519 and DS 2 are fully in-circuit; ECDSA-P256 has a Phase-4 v2 boundary binding the 0.1% residual to verifier-derivable values; RSA-2048 leaf signatures are in-circuit, while parent-link RSA at the root/TLD remains in $\mathcal{S}_{\mathrm{att}}$ pending in-circuit deployment of the same gadget.

Algorithm / digest	Set	Note
Ed25519 (alg. 15)	$\mathcal{S}_{\mathrm{ic}}$	full in-circuit
ECDSA-P256 (alg. 13)	$\mathcal{S}_{\mathrm{ic}}$	boundary-bound residual
RSA-2048 leaf (alg. 8)	$\mathcal{S}_{\mathrm{ic}}$	full in-circuit
RSA-2048 parent-link	$\mathcal{S}_{\mathrm{att}}$	same gadget pending deploy
DS type 2 (SHA-256)	$\mathcal{S}_{\mathrm{ic}}$	full in-circuit
DS type 1/4 (SHA-1/-384)	$\mathcal{S}_{\mathrm{att}}$	gadgets pending

This violates EUF-CMA security of ML-DSA, occurring with probability at most $\varepsilon_{\mathrm{sig}}$.

By the union bound, for any chain link in $\mathcal{S}_{\mathrm{ic}}$:

$$\Pr[r^* \text{ accepted}] \leq \varepsilon_H + \varepsilon_\pi + \varepsilon_{\mathrm{sig}} \leq 3 \cdot 2^{-\lambda}.$$

Links in $\mathcal{S}_{\mathrm{att}}$ are not covered by E_3; their integrity follows from native prover verification under T2. □

Corollary 1 (Boundary-bound residual)**.** *For algorithms in $\mathcal{S}_{\mathrm{ic}}$ whose verifier contains a residual native check (the 0.1% of ECDSA-P256 verification: Z^{-1} and $R.x \bmod n$ equality; the verifier-derived EM column for RSA-2048), the residuals are bound to verifier-derivable values via boundary constraints on the AIR's last row. Substitution either breaks the public-input hash binding (ε_H) or solves the residual check, which reduces to ECDLP for ECDSA and to EUF-CMA of RSA-PKCS#1 v1.5 (in the ROM) for RSA. The bound of Theorem 1 therefore extends unchanged at λ.*

a) Deployment coverage of $\mathcal{S}_{\mathrm{ic}}$.: Tab. II lists the current coverage; residual gaps (DS type 1/4, root/TLD-level RSA) are not material for the intended targets, where leaf zones are predominantly ECDSA- or Ed25519-signed and Theorem 1's in-circuit guarantee binds the leaf security boundary.

B. Properties 2–4: Authentication, Non-Substitutability, Freshness

Theorem 2 (Prover authentication)**.** *No adversary (classical or quantum) produces a well-bound epoch package without sk except with probability $\varepsilon_{\mathrm{sig}}$ (direct reduction to ML-DSA EUF-CMA).*

Theorem 3 (Non-substitutability)**.** *An adversary cannot bind a legitimate STARK proof π to a different Merkle tree $\mathcal{M}'$ ($R' \neq R$) except with probability $\varepsilon_H + \varepsilon_\pi$. R is a public input to π; substitution either fails STARK soundness or requires a hash collision $R = R'$ on $\mathcal{M} \neq \mathcal{M}'$.*

Theorem 4 (Epoch freshness)**.** *An adversary cannot cause acceptance of a stale epoch package, provided the device enforces monotonically increasing seq and epoch-window checks on T; both are public inputs to π and covered by σ, so modification invalidates both.*

C. Residual Risk Quantification

The pre-proof chain attack (§III-D) reduces to a single forged leaf RRSIG per zone: $\mathcal{A}$ recovers the ZSK via Shor and forges one RRSIG that verifies under the unmodified ZSK. The unmitigated cost is *one* Shor execution plus on-path injection during the epoch window; estimates for RSA-2048 place C_{Shor} at $\sim 10^3$–10^4 logical qubit-hours [17], with Ed25519 comparable.

With V BGP-AS-disjoint vantages and *idealised independence* of on-path probability p_{op}, joint forgery probability is bounded by p_{op}^V. We emphasise this is a *model-only* statement: real-world AS independence is imperfect (shared upstreams, common IXPs, operator correlation), so the effective V is strictly lower than the AS-count suggests. Quantifying the empirical residual via RIPE Atlas measurements + induced-tampering experiments is planned camera-ready / follow-on work (§VII-F); the model-only bound should not be cited as an operational guarantee until that validation lands. Temporal jitter further forces sustained presence across ΔT, raising the campaign cost. The hash-chained epoch log gives third-party auditors post-hoc detection of contradictions across epochs, sufficient to trigger out-of-band revocation in regulated deployments. Without STARK-DNS a quantum adversary can serve forged records to any victim indefinitely; with STARK-DNS plus the four mitigations the residual surface is the joint network-vantage and timing window. We claim no multiplicative cost across delegation levels: the gain is in the network-and-timing dimension, not the cryptographic-key dimension.

a) Consumer-side quantum-MITM is structurally unavailable after authentic-epoch installation.: The silent quantum-MITM baseline of §III-A—intercept the victim's DNS query, return a Shor-forged RRSIG pointing to an attacker-controlled server, leave no detectable artefact—is *structurally unavailable* against a STARK-DNS edge device once an authentic epoch package has been installed (see §III-D for the pre-proof window and §IV-B for bootstrap and update channels). The device issues no runtime DNS queries; it answers locally from the STARK-attested Merkle tree, and an adversary on any network path to it has no message to intercept and nothing to forge. Replacing a proven record requires either breaking the prover's ML-DSA key, or inducing the legitimate zone to actually publish a different record so that the change is captured in a *future* epoch's delta. The latter is a registered, publicly observable, audit-trail-creating action against the zone authority—not the silent UDP interception that classical DNSSEC permits. The adversary's bar moves from "MITM the next query" to "compromise the zone's publishing infrastructure or the prover's signing key" for every record already in the rolling commitment, and any successful tampering leaves traces detectable across epochs.

D. Swarm Prover Security

The single-prover deployment shape analysed above places the authority entity in a single trust anchor: any compromise of that prover's signing key allows an adversary to mint authentic-looking epoch packages containing forged records. For deployments where this concentration is unacceptable (`.gov`, `.mil`, hyperscale corporate), we extend the architecture to a *swarm* of provers operating under explicit Byzantine fault-tolerance assumptions.

a) Swarm roles.: The swarm consists of three role classes:

- **Primary controller** (C): orchestrates a proving job, holds an ML-DSA key pair, and signs the final epoch package. The primary's signing key is the analogue of the single-prover key in §IV-A.
- **Witness controllers** ($\mathsf{V}_1, \ldots, \mathsf{V}_N$): independent controllers operating in parallel. Each V_j has a distinct ML-DSA key pair and independently witnesses every shard's STARK proof, archiving its own *witness receipt* for each accepted shard.
- **Workers** ($\mathsf{W}_1, \ldots, \mathsf{W}_K$): a pool of provers, allowlisted (**E1**) by both the primary and each witness. Workers receive shard assignments, run the STIR-based prover, and return signed attestations.

b) Determinism collapses Byzantine checking to byte equality.: The STIR-based prover is deterministic [7], so two non-colluding workers assigned the same shard with the same Fiat–Shamir nonce produce byte-identical proofs; a Byzantine worker returning a different proof is trivially detected.

[Byzantine ensemble correctness] For any shard s assigned to r workers under the same $(jobId, nonce)$, if the primary accepts a single π_s and at least one honest worker participated, then π_s is the unique deterministic STIR output.

[M-of-N witness quorum] A verifier accepts the package only if $\geq M$ of N witnesses archive matching receipts for every shard, each signed under its distinct ML-DSA key. Forgery requires compromising $\geq M$ witness keys plus the primary, an $(M{+}1)$-out-of-$(N{+}1)$ key compromise across administrative boundaries within the epoch window.

A single C-key compromise therefore no longer suffices; the adversary must additionally compromise a witness quorum across organisational boundaries.

E. Formal Verification with Tamarin

The swarm protocol's symbolic security has been mechanically verified using the Tamarin prover [34]. The model captures ML-DSA signatures (Tamarin's `signing` builtin), hash digests (free function symbols, collision-resistant by construction), the allowlist gate, the worker/primary attestation flow, and the witness-archive flow. TLS confidentiality is intentionally NOT modelled—every wire message is sent in the clear and authentication is supplied by ML-DSA signatures, matching the network-adversary threat model of §III.

a) Adversary model.: The model exposes two adversary rules (`Compromise_Worker`, `Compromise_Witness`) capturing partial key compromise of allowlisted workers and witness controllers respectively.

b) Verified properties.: Table III summarises the ten lemmas mechanically proven by Tamarin. Each lemma corresponds to a numbered security property in the protocol design (see Appendix B).

TABLE III
TAMARIN-VERIFIED LEMMAS FOR THE SWARM PROTOCOL. ALL LEMMAS AUTO-PROVE IN < 1 S ON TAMARIN-PROVER 1.10.0 + MAUDE 2.7.1. STEPS REPORTED BY THE PROVER; *[reuse]* CHAINS FEED DOWNSTREAM LEMMAS.

Lemma	Status	Steps	Property
`proved_origin`	verified	7	*(source lemma)*
`protocol_executable`	verified	9	liveness
`allowlist_enforced`	verified	5	**E1**
`authority_binding`	verified	3	Property 1
`shard_inclusion`	verified	12	Property 2
`replay_freshness`	verified	2	Property 3
`deterministic_proving`	verified	2	Property 4
`receipt_provenance`	verified	2	*(auxiliary)*
`witness_quorum_2`	verified	8	Property 5 (M=2)
`witness_quorum_3`	verified	13	Property 5 (M=3)

The most informative lemmas are `shard_inclusion` (every accepted shard is backed by an honest worker's signed attestation modulo key compromise) and `witness_quorum_`M (verified explicitly for M=2, 3; the general M-of-N statement requires multiset induction outside Tamarin's reachability and is a known modelling limitation).

c) What the Tamarin model does NOT cover.: Three classes of property remain outside the symbolic proof:

- **Quantitative STARK soundness**: the symbolic `h/1` hash is unconditionally collision-resistant, whereas the real prover has a $2^{-\lambda}$ soundness error. Quantifying λ requires the modular IT-bind-FS decomposition of [7] with the STIR proximity-test bound [6].
- **Liveness in adversarial schedulers**: Tamarin's reachability semantics is not designed for fairness or DoS-resistance arguments; we believe these are best analysed in a probabilistic process algebra and leave them to follow-on work.
- **Pre-proof DNSSEC chain compromise**: this is a TOCTOU-class attack against the DNSSEC infrastructure the prover *reads*; it is orthogonal to the protocol the swarm runs and is analysed informally in §III-D.

The Tamarin theory (∼226 lines including the source lemma) is reproduced verbatim in Appendix B for independent re-verification.

d) Model–code correspondence.: We have manually audited the ten Tamarin rules against the prototype's controller, worker, protocol-message, and edge-verifier components. Each rule maps to a concrete code path: ML-DSA-65 (FIPS-204) keypair generation, SHA3-256 worker-allowlist enforcement, per-shard nonce assignment, the canonical shard_fs_binding feeding the STARK public-input hash, and ML-DSA-signed controller / witness receipts. The full mapping (crate names, signatures, byte-level domain-separation strings) is in Appendix B.

e) Soundness-preserving abstractions.: The Tamarin model is a strict under-approximation of the implementation: symbolic message tags, public-input bundles, and witness-role realisations all strengthen, never weaken, the binding captured in the lemmas (full mapping in Appendix B). The proof re-runs in 0.60 s wall-clock against the current code (`tamarin-prover` 1.10.0 + `maude` 2.7.1, all ten lemmas verified).

VI. IMPLEMENTATION AND EVALUATION

This section reports on a prototype implementation and empirical evaluation of the STARK-DNS architecture. Our primary goals are to establish computational feasibility, quantify the cost of in-circuit DNSSEC signature verification, measure end-to-end proof generation and verification times, and compare per-query resolution cost against classical DNSSEC.

A. Implementation

The prototype (Rust, ∼40k LOC, MIT-licensed; full source and benchmark harness submitted as supplementary material [8]) builds on the FIPS-aligned framework of [7] with STIR [6] as the low-degree test (Level 3: Goldilocks trace, $\mathbb{F}_{p^6}$ commitments, SHA3-512, STIR arity k=4, r=79 rounds). Four components: (i) a DNSSEC resolver and chain capturer (`hickory-dns` [20] based), (ii) AIR encoders for the three signature schemes, (iii) the STARK prover/verifier, and (iv) ML-DSA-65 signer + SHA3-512 Merkle tree (Level 3). AIR encoders (with measured constraint counts in parentheses): Ed25519 v16 verify-AIR ($\sim 7.7\times10^7$ cons, fully in-circuit RFC 8032 §5.1.7 cofactored predicate); ECDSA-P256 double-chain AIR ($\sim 4\times10^7$ cons, K=256 scalar-mult chain under Renes–Costello–Batina 2016 with v2 boundary binding to verifier-derived r_{proj} columns); RSA-2048 exponentiation AIR ($\sim 3.5\times10^5$ cons, 17-step e=65,537 chain with row-16 boundary binding to verifier-derived EM column). Reproducibility: every measurement reported in this section is produced by a `cargo run --release -p swarm-dns --example` command named in the corresponding paragraph; 35+ runnable examples + 545 unit tests in `crates/deep_ali` cover the full AIR surface.

B. Experimental Setup

a) Hardware.: Prover benchmarks: AWS Graviton4 (Neoverse-V2, DDR5-5600) on `r8g.4xlarge` (16 vCPU, 128 GiB) as canonical, with `c8g.4xlarge` (16 vCPU, 32 GiB) for the thread sweep (§VI-D); Apple M4 Mac mini as developer reference. Edge/verifier: Apple M4 and Raspberry Pi 4 (Cortex-A72, 1.8 GHz, 4 GB) represent constrained hardware.

b) Test zones.: Three synthetic configurations built from public zone samples (generator in the artefact): SMALL (N=1,000 `.gov` records), MEDIUM (N=10,000 `.gov`/`.mil`), LARGE (N=100,000 across multiple TLDs).

C. Per-signature STARK costs

All three production DNSSEC signing algorithms (RSA-2048, Ed25519, ECDSA-P256) prove end-to-end on a single AWS Graviton4 `r8g.4xlarge` (16 vCPU, `RAYON_NUM_THREADS`=16). We use canonical test vectors where applicable (RFC 6979 §A.2.5 for ECDSA; RFC 8032 §5.1.7 cofactored Ed25519 verify; deterministic synthetic

TABLE IV
SINGLE-PROVER WALL TIME (S) ON AWS `c8g.4xlarge` (16 VCPU, DDR5-5600) AS T=`RAYON_NUM_THREADS` SWEEPS 4→16; PEAK RSS IN GIB. THE `ed25519_streaming_merge` ROW REPORTS THE FULL $K=\{8, 32, 64, 128, 256\}$ SWEEP (USED TO CHARACTERISE STREAMING-MERGE BANDWIDTH SCALING); THE CANONICAL ED25519 SINGLE-SIGNATURE TIME AT K=256 IS 105 S ON `r8g.4xlarge` (§VI-C). `crossalg_3sig` AGGREGATES ALL THREE PRODUCTION ALGORITHMS BACK-TO-BACK AT K=256.

Benchmark	T=4	T=8	T=12	T=16	RSS
`ecdsa_record` (K=256 single)	107	98	96	95	20.3
`ed25519_streaming_merge`	280	264	259	255	31.1
`crossalg_3sig`	219	206	202	200	21.0
`mixed_zone_rollup`	219	207	203	201	21.0
`dns_chain_multirec`	881	811	795	805	—

keypair for RSA round-tripped against the `rsa` crate). Per-signature wall time and proof size:

Algorithm	Cons. (total)	Per-sig prove	Proof bytes
RSA-2048	$\sim 3.5\times10^5$	**0.68 s**	99 KiB
Ed25519 (K=256)	$\sim 7.7\times10^7$	**105 s**	138 KiB
ECDSA-P256	$\sim 4\times10^7$	**95 s**	138 KiB

RSA is the cheapest in-circuit despite its 2,048-bit modulus, by virtue of e=65,537's small Hamming weight (17 multiplications versus Ed25519's 256-bit ladder). Per-record cost is empirically constant across multiple records ($\leq$ 3% variance, N=10 for RSA, N=2 for Ed25519/ECDSA; `dns_chain_multirecord_bench`); the prove pipeline parallelises linearly across a worker swarm. A unified single-run reproducibility harness (`crossalg_three_signature_bench`) reproduces all three measurements back-to-back with all proofs verifying locally. The workload is memory-bandwidth-bound: phase breakdowns (Section A) show LDE expansion dominates $90-95\%$ of wall time across all three algorithms, confirming the bottleneck. Per-instance scaling is characterised in §VI-D.

D. Single-instance scaling and deployment cost

We characterise prover scaling on AWS Graviton4 by sweeping `RAYON_NUM_THREADS` from 4 to 16 on a single `c8g.4xlarge` instance across five benchmarks (Tab. IV). All five scale monotonically; speedup from T=4 to T=16 is 8–11%, indicating a $\sim$4-core bandwidth ceiling. STARK-DNS therefore scales *horizontally* (across independent instances) rather than vertically.

a) Concurrent provers and cost parity.: Co-locating W parallel ECDSA provers on `r8g.4xlarge` (16 vCPU, 128 GiB, $\sim$ \$0.28/hr spot [1]) at $\lfloor 16/W \rfloor$ Rayon threads each yields per-worker efficiency $\eta_{W=2}$=0.96, $\eta_{W=4}$=0.88 ($\eta = t_{\text{single}}/t_{\text{agg}}$); W=8 exceeds the 128 GiB capacity ($\sim$ 20 GiB/worker, 8×20=160 GiB) and is OOM-killed — the practical ceiling is W=6. ECDSA-P256 records cost $\$0.07 \cdot 95/3600 \approx \$0.00185$/proof on `r8g.xlarge` (extrapolated from the same Graviton4 cores at $\sim$ \$0.07/hr), and $\$0.28 \cdot 27/3600 \approx \$0.00210$/proof on `r8g.4xlarge` at W=4 — approximate cost parity at 4× per-instance throughput. Both shapes converge to $\sim$ 480 ECDSA proofs per dollar-hour; operators choose between many small fault-isolated instances and denser concurrent-prover deployments at the same cost. The TLD-scale projection of Tab. V ($\sim$ \$3,010 for the `.com` signing pass on a 1,000-worker swarm) follows directly. We rule out the compute-optimised `c8g` family (2 GiB/vCPU): it OOM-kills at W>1 for ECDSA and any W for Ed25519 (31 GiB peak RSS). The full sweep is reproducible via `scripts/bench_thread_sweep.sh`.

b) Worker definition.: A "worker" is one `r8g.xlarge`-equivalent prover process (4-vCPU Graviton4, T=4); a 1,000-worker swarm is 1,000 `r8g.xlarge` or 250 `r8g.4xlarge` at W=4. Tab. V uses the RSA-2048 sharded rate ($\sim$ 1 s/record); sharded ECDSA/Ed25519 measurements are pending camera-ready.

E. Edge-consumer artefact and verify cost

The architecture supports four consumer-facing artefact shapes: (A) full audit, (B) light swarm, (C) stacked-AIR, and (D) sharded rollup (detailed below). Modes A–C measured on N=100 RSA records (extrapolate linearly), AWS `r8g.4xlarge` prover with M4 / Raspberry Pi 4 edge verifier, real FIPS-204 ML-DSA-65:

Mode	Size	Verify	Trust assumption
A. Full audit	10.5 MiB	51.3 ms	STARK + ML-DSA
B. Light (swarm)	138 KiB	1.56 ms	+ swarm honesty
C. Stacked AIR	**100 KiB**	**1.5 ms**	STARK + ML-DSA

Option C is strictly better than B for sectoral deployments (one signing CA per zone): identical consumer-side property as STARK-of-STARK recursion (constant size, full STARK soundness, no auditor-trust), achieved by stacking N independent RSA exp chains at disjoint column ranges of a single wide trace. The STIR proof size is constant in N (the per-record AIR's n_{trace}=32 is unchanged; only trace width grows). Trade-off: prover work is sequential within one trace ($O(N)$, no swarm-prover parallelism); recursive STARK is the principled-soundness endpoint that allows both swarm-prover parallelism and constant edge artefact ($\sim 10^5$–10^6 constraints/inner-proof to encode the STIR verifier; deferred).

a) Edge-consumer end-to-end + airgapped resolution (measured).: A complete edge flow with real cryptography (`edge_consumer_full_demo` / `airgapped_resolver_demo`, N=50): authority ML-DSA-65-signs $(\mathsf{stir_commitment}, \mathsf{merkle_root}, \mathsf{pk}_{\mathsf{zsk}}, \mathsf{seq}, T)$, bundle is 118 KiB. Edge boots network-free in 868 μs (86 μs ML-DSA + 672 μs STARK + parse), then serves 1,000 random queries in 1.16 ms cumulative (1.156 μs/query). Bundle distribution is channel-agnostic (CDN, IPFS, USB, satellite, LoRa, QR code).

b) TLD-scale sharded architecture (measured + projected).: Above $\sim$10k records the stacked-AIR pattern hits a memory ceiling. The zone is partitioned into K shards of S records, each running an independent stacked-AIR STARK whose π_{hash} binds the global Merkle root; an outer-rollup STARK commits the K shard π_{hash}es, and ML-DSA-65 signs

TABLE V
TLD-SCALE PROVE TIME, SHARDED ARCHITECTURE (MEASURED + PROJECTED, K=4 × S=50 BASELINE). THE EDGE CONSUMER ARTEFACT STAYS ∼89 KIB PLUS A $\log_2 N{\cdot}64$ B MERKLE PATH REGARDLESS OF N.

Zone	Records	Single machine	1,000-worker swarm
`.gov` / `.mil`	10 k	2.7 h	12 s
`.uk`	10 M	113 days	2.7 h
`.com`	160 M	1,801 days	43 h

(outer STIR root, global Merkle root, $\mathsf{pk}_{\mathrm{zsk}}$, seq, T). We measure $K{=}4 \times S{=}50$ (`tld_sharded_prover_demo`): each shard proves in 48.6 s ($\sim$1 s/record empirical at $S{=}50$, amortising LDE/streaming-merge across the wide trace — per-record constraints unchanged), outer-rollup adds 1.50 ms prove + 0.45 ms verify; edge artefact is 89 KiB, once-per-epoch verify 729 μs, per-query Merkle verify 1.46 μs. Per-shard cost is constant in S: The edge artefact stays at $\sim$ 89 KiB plus a $\log_2 N{\cdot}64$ B Merkle path ($\approx$ 1.75 KiB at .com scale). Per-query consumer verify scales as $O(\log_2 N)$ hash invocations on M4: $\approx$0.4–0.5 μs at $N{=}$10k, $\approx$1.5–1.7 μs at .com scale. A full re-prove of `.com` would require $\sim$43 h of aggregate worker time (parallelisable across 1,000 workers); steady-state operation re-proves only the inter-epoch delta, not the full zone (§VII-D). Smaller TLDs (`.gov`, `.mil`) re-prove in seconds. Inner-proof artefacts for full-audit mode total $K \cdot 97$ KiB (15 GiB at .com scale, distributed via CDN). The edge consumer's trust ladder is explicit across the four deployment modes: full-audit (A) and stacked-AIR (C) preserve *full* STARK soundness without swarm or auditor trust; light-swarm (B) preserves per-record STARK soundness but requires honest-majority swarm trust; sharded (D) preserves per-shard STARK soundness and additionally trusts that each shard's π_{hash} correctly binds the global Merkle root, closed either by out-of-band auditor re-verification or, in the principled-soundness endpoint of the architecture, by full STARK-of-STARK recursion.

c) Quantum-MITM attack: STARK-DNS vs DNSSEC (measured/demonstrated).: We exercise the silent-quantum-MITM threat model end-to-end (`quantum_mitm_attack_demo`): the adversary recovers the zone's RSA-2048 private key (Shor's algorithm; here we just hand the adversary the same key) and runs three concrete attack flows against an edge device that has bootstrapped its trust on the legitimate authority's ML-DSA-65 public key.

Attack flow	DNSSEC verifier	STARK-DNS edge verifier
(A) Forge RRSIG re-signing `rec0042` to a malicious IP (6.6.6.6)	**ACCEPT** *	**REJECT**
(B) Sign counterfeit epoch package with adversary's ML-DSA key	n/a	**REJECT**
(C) Forge SHA3-512 collision against pinned Merkle root	n/a	**REJECT**

* Native RSA verify under the now-broken zone key returns "valid signature"; DNSSEC routes the user to 6.6.6.6 with no detection.

STARK-DNS rejects all three: (A) the forged leaf does not chain to the device's pinned Merkle root (would need a SHA3-512 2nd-preimage, $\geq 2^{256}$ quantum work); (B) the counterfeit package is signed under the adversary's ML-DSA key, not the authority's pinned key (would need to break ML-DSA-65, lattice-based PQ-secure); (C) a direct collision on the 512-bit pinned root is computationally infeasible. The architectural property: **classically-signed RRSIGs become PQ-protected, for edge consumers that install an authentic epoch package, from the moment they are committed under an ML-DSA-signed STARK-DNS epoch root forward, even when originally signed under classical keys**; edge consumer trust rests on (a) the authority's ML-DSA-65 public key and (b) SHA3-512 collision resistance, both PQ-secure today.

d) Harvest-now, protect-later (HNPL).: We name the property above *harvest-now-protect-later*, the time-symmetric dual of HNDL: where HNDL collects classical signatures today to break post-CRQC, HNPL commits them into an ML-DSA-signed STARK-DNS epoch today, foreclosing adversarial harvest of records sealed before t_0 whenever t_0 precedes the relevant collection window. An authority running a genesis ceremony at $t_0 \ll t_{\mathrm{CRQC}}$ obtains PQ authenticity for pre-t_0 records, a PQ-only update path post-t_0, and network-independent edge resolution — all at the bounded cost of §VI-D, none of it requiring the PQ-DNS standardisation timeline.

e) Incremental epoch updates.: The prototype's *epoch-stable chain FS / epoch-fresh outer FS* layout (FS = Fiat–Shamir transcript; defined formally in §VII-D) lets unchanged records reuse their chain bundles byte-for-byte across epochs; only modified records and the outer-rollup STARK need re-proving. The per-record statement bound by π_i is $(\mathsf{rrset}_i, \mathsf{rrsig}_i, \mathsf{dnskey}_i, [\mathsf{ds}_i, \mathsf{parent_dnskey}_i, \ldots],$ $\mathsf{inception}_i, \mathsf{expiration}_i)$ in canonicalised wire format; the chain Fiat–Shamir transcript is keyed only by these bytes (it does *not* include the epoch sequence number seq or epoch timestamp T), making π_i epoch-stable. The outer-rollup statement, keyed by $(\mathsf{seq}, T, \mathsf{revocation_root})$, binds every reused π_i's public-input hash plus the current revocation root into the new epoch root R. An edge or auditor accepts a reused π_i at epoch T iff (a) T lies within $[\mathsf{inception}_i, \mathsf{expiration}_i]$ and (b) rrset_i is not in the revocation list signed under the outer rollup. For a zone with 1% daily churn, daily refresh on a 1,000-worker swarm at $N = 10{,}000$ takes $\sim$6 min wall-clock for changed records plus seconds for the outer rollup. This is the deployment shape we recommend: full epoch on day 0, incremental refresh thereafter.

f) Per-query latency vs DNS / DNSSEC.:

Method	Latency	Network	PQ-safe
Standard DNS	10–100 ms	Yes	No
DNSSEC	20–200 ms	Yes	No
STARK-DNS	1–2 μs	**No**	**Yes**
Local cache	<0.1 μs	No	No

STARK-DNS's per-query cost is orders of magnitude below online DNSSEC validation in steady state, eliminates the network dependency, and gains post-quantum security. A worst-

TABLE VI
IN-CIRCUIT SIGNATURE VERIFICATION COST (MEASURED END-TO-END, $K{=}256$, SINGLE STARK PER SIGNATURE). SHA-256 ROW IS THE STANDALONE SHA256_AIR MEASUREMENT AT SINGLE-BLOCK INPUT.

Algorithm	Cons. (total)	Prove time per sig
RSA-2048-SHA256 (PKCS#1 v1.5)	$\sim 3.5{\times}10^5$	0.68 s
ECDSA P-256 (RFC 6979 §A.2.5)	$\sim 4{\times}10^7$	95 s
Ed25519 (RFC 8032 §5.1.7)	$\sim 7.7{\times}10^7$	105 s
SHA-256 (DS type 2, AIR)	766 cons/row $\times$ 128 rows	$<$ 100 ms

case policy re-verifying the full epoch on every query still pays only $\sim$1 ms.

F. In-circuit signature soundness models

Ed25519 is fully in-circuit (no auxiliary verifier work). **ECDSA-P256** runs the double scalar-mult $u_1{\cdot}G + u_2{\cdot}Q$ in-circuit; a boundary constraint at row $K{-}1$ binds the chain output (projective R_a, R_b) to verifier-supplied r_{proj} columns committed via π_{hash}, and the verifier deterministically replays the same RCB-2016 chain natively to derive those values plus the trailing Z^{-1} + $R.x \bmod n$ equality (deterministic EC/modular arithmetic, no key material). **RSA-2048** runs the $s^{65,537} \bmod n$ exponentiation chain in-circuit (17 multiplications), with a row-16 boundary constraint binding the chain output to a verifier-derived EM column ($EM = \mathrm{pkcs1_pad}(\mathrm{SHA\text{-}256}(m), k{=}256)$); the verifier's SHA-256 + byte-constant work is trivially auditable. A v3 layout bringing the residual verifier-side work (ECDSA's affine conversion; RSA's SHA-256 + PKCS#1) fully in-circuit is identified as future work; current soundness is sufficient for production deployment under "trust the DNS authority's ML-DSA epoch key".

VII. DISCUSSION

A. Prover Trust and Decentralisation

The prover is CA-analogous; our recommended mitigation is the swarm protocol of §V-D (primary controller + M-of-N witness controllers, Tamarin-verified in Appendix B). Sectoral deployments (`.gov`, `.mil`) match a single in-house prover, with publicly verifiable STARKs allowing any third party to cross-check outputs against independent DNSSEC resolution.

B. Epoch Duration and Freshness

Epoch duration ΔT trades freshness against prover cost. For the target deployments (`.gov` typical SOA refresh 3,600 s, minimum TTL 300 s), a 1-hour epoch aligns with existing refresh cycles and produces records no more stale than standard DNS caching. Highly dynamic zones (e.g., CDN load balancing) are out of scope; the architecture targets zones whose records are stable over hours or days, which covers the vast majority of critical infrastructure DNS. The default edge policy enforces epoch freshness only ($T \in [\mathsf{inception}, \mathsf{expiration}]$ at proof time); deployments MAY enable an optional TTL$\wedge$epoch policy that further restricts a record's serving lifetime to $\min(\mathit{TTL}, \Delta T)$ at lookup time (§IV-F).

C. Incremental Deployment

a) Non-disruptive complementary wrapper.: STARK-DNS is purely additive: authoritative nameservers, recursive resolvers, registrars, and the DNS wire format are unchanged. The prover consumes existing classical DNSSEC output and produces a post-quantum wrapper around it, delivering immediate quantum-safe authentication for *records signed under RSA, ECDSA, or Ed25519 today*, without waiting for any zone, TLD, or root operator to migrate their signing infrastructure. Each adopting organisation flips the switch unilaterally for its own edge devices; DNSSEC operators it depends on need not coordinate.

b) Sidesteps the PQ-DNS UDP-size problem.: The principal obstacle to in-band PQ-DNSSEC migration is packet size: an ML-DSA-65 three-level chain alone is $\approx$9,879 bytes (Table I, §II-B), forcing TCP fallback or fragmentation for nearly every query and breaking middleboxes that assume UDP/EDNS0 1,232-byte responses [15], [18], [28]. STARK-DNS avoids this entirely: the post-quantum cryptographic objects (ML-DSA signature and STARK proof) ride on the *epoch package distribution channel* (CDN, HTTPS, broadcast, physical media—any out-of-band path) rather than on the DNS wire. Per-query DNS traffic remains classical DNSSEC at its existing packet sizes; the PQ heavy-weight artefacts move once per epoch through a channel where multi-MiB payloads are routine and unproblematic.

Edge devices with network connectivity may fall back to standard DNSSEC at any time, so the offline capability is a supplement rather than a replacement.

D. Handling Zone Updates

a) Delta-only steady state.: Each subsequent epoch's outer rollup aggregates existing per-record proofs into the new Merkle commitment without re-running the inner STIR-based prover (FS = Fiat–Shamir transcript throughout), so per-epoch work covers only the delta: records added, updated, due for re-proof because their RRSIGs are about to expire, or placed on the revocation list. For typical critical-infrastructure zones the delta per epoch is $<1\%$ of the zone, reducing per-epoch prover work by two or more orders of magnitude relative to a from-scratch reproof.

b) Epoch overlap and emergency revocation.: Edges accept records from current and next epoch during a configurable overlap window for non-disruptive rollout. Emergency updates use an ML-DSA-signed revocation list checked alongside the Merkle tree (CRL/OCSP analogue); revoked records are excluded from the next epoch's rollup.

E. Scalability

The STARK proof itself is polylogarithmic in N ($O(\log^2 N)$ for STIR), so it does not grow meaningfully with zone size; the per-record edge artefact is the STARK proof plus a $\log_2 N$-deep Merkle inclusion path ($\sim 64 \log_2 N$ bytes) and the once-per-epoch ML-DSA signature. The full Merkle tree is linear: for $N = 10^6$ records it requires $\approx$128 MiB on the prover side (64-byte nodes, $\approx 2N{-}1$ total); edge devices

need only the $O(n \log N)$ nodes covering their n relevant leaves. Prover cost is linear in N and dominated by in-circuit signature verification; swarm parallelism keeps wall-clock manageable (Table V), and recursive proof composition can reduce amortised cost further at the expense of implementation complexity. The dollar cost is linear in cloud pricing: even a $10\times$ spot-price increase keeps the `.com` signing pass within five-figure totals, and on-premise deployment is bounded by the same per-proof core-hour count (§VI-D).

F. Limitations

Dynamic DNS. The architecture cannot handle records that change per-query (e.g., some CDN load-balancing responses); it explicitly targets zones whose operational model already assumes TTL-scale stability, and we make no claim of applicability to latency-optimised content distribution frontends. Longitudinal TTL measurements report median TTLs of hours to days for institutional and government zones, in contrast to CDN-fronted zones where sub-minute TTLs predominate [27]; daily or sub-daily epoch refresh (§VII-D) absorbs the inter-epoch delta for the targeted regime.

Prover computation. In-circuit verification of classical signatures is expensive in absolute terms (Ed25519 and ECDSA-P256 measured at $\sim 1.5-1.7$ min per signature on `r8g.4xlarge`; RSA-2048 at 0.68 s/sig with our e=65,537 exp-chain gadget — naïve in-circuit arithmetisation can be much slower). Practical deployment at TLD scale benefits from sharded amortisation (§VI-E) and, eventually, recursive proof composition.

Single prover trust. Without federation, the prover is a single point of trust (though not a single point of failure—edge devices fail open to standard DNS if the prover is unavailable).

No formal FIPS module. The construction uses FIPS-approved primitives (SHA-3, ML-DSA) but is not itself a CMVP-validated module. Validation is an engineering effort outside the scope of this paper.

Multi-vantage measurement. The p^V_{op} independence model in §V-C is not yet empirically backed; validation via AS-disjoint RIPE Atlas probes, cross-vantage byte-equality on returned RRSIGs, and a controlled on-path tampering experiment is planned follow-on work.

Authenticated denial-of-existence (NSEC/NSEC3). The current construction authenticates queries against the *committed* RRsets only; it does not authenticate non-existence under DNSSEC's NSEC/NSEC3 semantics, including wildcard synthesis, NXDOMAIN-vs-NODATA distinctions, and NSEC3 opt-out. Offline resolution claims throughout the paper apply to queries whose answer is a committed RRset; queries falling outside the commitment fail open to standard DNS resolution. In-circuit encoding of NSEC/NSEC3 with wildcard synthesis is design-level future work; an empirical study of the fraction of queries in our target environments (`.gov`/`.mil`/critical-infrastructure) that require denial-of-existence semantics will accompany that extension.

Privacy and zone enumeration. A full epoch dictionary enumerates zone names by construction (AXFR-equivalent posture). Operators preferring confidentiality combine per-device subtrees (§IV-B) with salted name hashes at the leaf, mirroring NSEC3's trade-off; public-zone operators ship full dictionaries without regression.

Soundness parameters. At Level 3 (λ=192, $q \leq 2^{65}$, SHA3-512, K=256, STIR k=4, r=79), the modular bound gives $\varepsilon_\pi \leq 2^{-194} \cdot 3 < 2^{-192}$ ([7], Tab. 5).

VIII. Related Work

A. DNSSEC Post-Quantum Migration

The IETF DNS Operations Working Group has explored several approaches to post-quantum DNSSEC [23]. Active drafts consider direct substitution of DNSSEC signature algorithms with PQC algorithms, hybrid classical/PQC signatures, and hash-based signatures (XMSS, SLH-DSA); Sheth [35] sketches a phased deployment strategy combining algorithm rollover, EDNS-buffer size negotiation, and TCP fallback — assuming coordinated authoritative-server, resolver, and registrar updates over a multi-year timeline. All of these approaches require modifications to authoritative nameservers, resolvers, and in some cases the DNS wire format; STARK-DNS by contrast requires no such coordination because the post-quantum artefact never traverses the DNS wire format. Goertzen and Stebila [18] propose ARRF, an application-layer *request-based fragmentation* mechanism delivering large PQ signatures inside DNSSEC responses without forcing TCP fallback; ARRF still requires resolver and authoritative-server changes to fragment and reassemble fragments at the application layer. Müller et al. [28] measure the performance impact of PQC signatures on DNSSEC validation latency. Our approach is orthogonal: we do not modify the DNS infrastructure at all, but instead wrap its output in a post-quantum verification layer.

B. DNS over HTTPS/TLS, VC for Infrastructure, and Offline DNS

DoH [21] and DoT [22] protect the stub-to-recursive transport channel but neither replace authoritative-side RSA/ECDSA nor work without network connectivity. STARKs and SNARKs have been applied to infrastructure verification (zkTLS [37], blockchain rollups [13]); our application to DNSSEC chain-of-trust is novel in that the proof attests to data *provenance*, not just integrity. Prior offline DNS work (local caching, AXFR/IXFR [25], DNS bundles) provides no cryptographic guarantee that survives the DNSSEC signatures' vulnerability window; our construction adds post-quantum authenticated offline resolution with $\sim$20 hashes per query.

C. Why not simply sign a Merkle tree?

A natural minimal alternative is to run a conventional DNSSEC-validating resolver, build a Merkle tree over the resulting records, and sign the root with a post-quantum signature; edge consumers verify the signature and inclusion proofs without performing DNSSEC validation themselves. This delivers integrity and PQ-authentication of the dataset, but it shifts the security argument from *cryptographic* to

TABLE VII
SIGNED MERKLE TREE VS. STARK-DNS.

Property	Merkle	STARK-DNS
PQ-authenticated dataset	✓	✓
Compact inclusion proofs	✓	✓
Resolver-trust eliminated	—	✓
Provable validation correctness	—	✓
Publicly verifiable record provenance	—	✓

operational: edge consumers must trust the resolver operator to (i) implement DNSSEC validation correctly, (ii) execute it faithfully, and (iii) admit no records that fail validation or violate operator policy. None of these is publicly verifiable. STARK-DNS replaces those operational assumptions with a STARK proving the records are the deterministic output of the DNSSEC validation constraints under the globally recognised root KSK at epoch time T (Tab. VII); the proof cost ($\sim$\$0.002/record, §VI-D) is the explicit price paid to remove operator trust from the trust chain.

a) Precedent: Certificate Transparency.: The structural critique is the same one that motivated Certificate Transparency in the TLS PKI [24]: operators are *trusted* but not *verified*, and misconfiguration is undetectable. The empirical gap in DNSSEC is large: Chung et al. measured $\sim$31% of DNSSEC-signed second-level domains as misconfigured [14], and APNIC measures only $\sim$30% of users behind validating resolvers [2]. STARK-DNS is to DNSSEC what CT is to the TLS-CA PKI: an operator-independent transparency layer that closes the same trust gap. A signed Merkle tree suffices when trusting the operator is part of the threat model (e.g., a single-tenant CDN signing its own zone for its own edge nodes); STARK-DNS targets the complementary setting in which eliminating operator trust is a security requirement.

b) Why not threshold ML-DSA, witness cosigning, or reproducible builds?: These non-STARK alternatives all attest to a quorum's *belief* about the validation result rather than the validation function itself: threshold ML-DSA signs whatever $M \leq N$ resolvers agree on, witness cosigning attests to artefact observation, reproducible builds attest to software identity. M colluding parties in any of these schemes produces a valid attestation over arbitrary records. STARK-DNS uniquely binds the validation *function* to its output in a publicly checkable object.

D. FIPS-Aligned STARKs

Our STARK construction adapts the modular FIPS-aligned framework of [7], which decomposes post-quantum soundness as $\varepsilon \leq \varepsilon_{\mathrm{IT}} + \varepsilon_{\mathrm{bind}} + \varepsilon_{\mathrm{FS}}$ (information-theoretic, hash-binding, and Fiat–Shamir terms), each tunable independently of the target security parameter λ. The framework instantiates the IT term with the BCIKS proximity-gap bound [11] for binary-folding FRI [9], [12]; we substitute STIR [6] as the low-degree test, which delivers smaller proofs and faster verification at equivalent IT-soundness. The hash-binding and Fiat–Shamir terms transfer verbatim, since they depend only on the SHA-3 family and the multi-round Fiat–Shamir compiler, not on the underlying proximity test. Concrete parameters follow the same Goldilocks $\mathbb{F}_{p^6}$ instantiation; STARK-DNS targets NIST post-quantum Level 3 (λ=192, $q \leq 2^{65}$).

IX. CONCLUSION

The post-quantum migration of DNSSEC is an urgent problem without an Internet-scale deployment path that preserves existing operational assumptions. In-band candidates — TCP-by-default, DoH, larger EDNS0 buffers — all relax one or more of these assumptions, and the required infrastructure changes span every component of the DNS ecosystem.

We have presented STARK-DNS, an architecture that provides an alternative deployment path independent of in-band PQ migration, by wrapping the existing DNSSEC infrastructure in a post-quantum verification layer. The construction binds prover identity, data integrity, data provenance, temporal validity, and efficient lookup capability into a single cryptographic object—a STARK proof whose public inputs include the Merkle root of the verified records and whose output is signed with an ML-DSA post-quantum signature. Edge devices verify the proof once per epoch and thereafter resolve DNS queries offline via Merkle inclusion proofs, reducing per-query cost from multiple classical signature verifications plus network round-trips to approximately 20 hash computations with no network access and stronger post-quantum security once an authentic epoch is installed.

The architecture requires no changes to authoritative nameservers, recursive resolvers, registrars, or the DNS wire format. It can be deployed incrementally, by individual organisations, for their own edge devices, today. For `.gov`, `.mil`, and national critical infrastructure zones that cannot wait for the decade-long DNS PQC migration, STARK-DNS provides immediate post-quantum protection with the operational bonus of offline resolution in communications-denied environments.

The construction is not without limitations. The prover is a trusted entity whose compromise would undermine the system; federation and public auditability mitigate but do not eliminate this concern. In-circuit verification of classical DNSSEC signatures is computationally expensive, and practical deployment at scale will likely require proof recursion or restriction to ECDSA-signed zones. The architecture is inappropriate for highly dynamic DNS zones. These are engineering challenges that we believe are tractable given current trends in prover optimisation, and the security and operational benefits justify the investment.

Succinct proofs of infrastructure correctness are an under-explored design pattern: the same architecture wrapping DNSSEC could in principle wrap any chain-of-trust verification (certificate transparency, BGP route attestations, software-supply-chain signatures). STARK-DNS is a concrete, open-source instantiation demonstrating the approach is practical.

REFERENCES

[1] Amazon Web Services. Amazon EC2 Spot Instance Pricing. https://aws.amazon.com/ec2/spot/pricing/. Accessed 2026-05-02; Gravi-

ton4 `r8g.4xlarge` us-east-1 spot, ∼$0.28/hr; `r8g.xlarge`, ∼$0.07/hr.

[2] APNIC Labs. DNSSEC validation rate by country. https://stats.labs.apnic.net/dnssec. Continuously updated; accessed 2026-05-02.

[3] R. Arends, R. Austein, M. Larson, D. Massey, and S. Rose. DNS Security Introduction and Requirements. RFC 4033, March 2005. URL: https://www.rfc-editor.org/rfc/rfc4033.

[4] R. Arends, R. Austein, M. Larson, D. Massey, and S. Rose. Protocol Modifications for the DNS Security Extensions. RFC 4035, March 2005. URL: https://www.rfc-editor.org/rfc/rfc4035.

[5] R. Arends, R. Austein, M. Larson, D. Massey, and S. Rose. Resource Records for the DNS Security Extensions. RFC 4034, March 2005. URL: https://www.rfc-editor.org/rfc/rfc4034.

[6] Gal Arnon, Alessandro Chiesa, Giacomo Fenzi, and Eylon Yogev. STIR: Reed–solomon proximity testing with fewer queries. IACR Cryptology ePrint Archive, Report 2024/390, 2024. URL: https://eprint.iacr.org/2024/390.

[7] Anonymous Authors. FIPS-Aligned STARKs with Concrete Multi-Level Post-Quantum Security. In *Anonymised for double-blind review*, 2026. Concurrent submission; cited under double-blind anonymisation.

[8] Anonymous Authors. STARK-DNS reference implementation, 2026. MIT-licensed open-source implementation; URL anonymised via anonymous.4open.science for double-blind review. URL: https://anonymous.4open.science/r/stark-dns-7415/.

[9] Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, and Michael Riabzev. Fast Reed–Solomon Interactive Oracle Proofs of Proximity. In *Proceedings of the 45th International Colloquium on Automata, Languages, and Programming (ICALP)*, volume 107 of *LIPIcs*, pages 14:1–14:17, 2018. https://doi.org/10.4230/LIPIcs.ICALP.2018.14 doi:10.4230/LIPIcs.ICALP.2018.14.

[10] Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, and Michael Riabzev. Scalable, Transparent, and Post-Quantum Secure Computational Integrity. IACR Cryptology ePrint Archive, Report 2018/046, 2018. URL: https://eprint.iacr.org/2018/046.

[11] Eli Ben-Sasson, Dan Carmon, Yuval Ishai, Swastik Kopparty, and Shubhangi Saraf. Proximity Gaps for Reed–Solomon Codes. In *Proceedings of the 61st Annual IEEE Symposium on Foundations of Computer Science (FOCS)*, pages 900–909. IEEE, 2020. https://doi.org/10.1109/FOCS46700.2020.00088 doi:10.1109/FOCS46700.2020.00088.

[12] Eli Ben-Sasson, Lior Goldberg, Swastik Kopparty, and Shubhangi Saraf. DEEP-FRI: Sampling Outside the Box Improves Soundness. In *Proceedings of the 11th Innovations in Theoretical Computer Science Conference (ITCS)*, volume 151 of *LIPIcs*, pages 5:1–5:32, 2020. https://doi.org/10.4230/LIPIcs.ITCS.2020.5 doi:10.4230/LIPIcs.ITCS.2020.5.

[13] Vitalik Buterin. An Incomplete Guide to Rollups, 2021. URL: https://vitalik.eth.limo/general/2021/01/05/rollup.html.

[14] Taejoong Chung, Roland van Rijswijk-Deij, Balakrishnan Chandrasekaran, David Choffnes, Dave Levin, Bruce M. Maggs, Alan Mislove, and Christo Wilson. A longitudinal, end-to-end view of the DNSSEC ecosystem. In *26th USENIX Security Symposium*, pages 1307–1322, 2017. URL: https://www.usenix.org/system/files/conference/usenixsecurity17/sec17-chung.pdf.

[15] J. Damas, M. Graff, and P. Vixie. Extension Mechanisms for DNS (EDNS(0)). RFC 6891, April 2013. URL: https://www.rfc-editor.org/rfc/rfc6891.

[16] DNS Flag Day Collaborators. DNS Flag Day 2020: Recommended minimum EDNS buffer size. https://dnsflagday.net/2020/, 2020. Accessed 2026-05-03.

[17] Craig Gidney and Martin Ekerå. How to Factor 2048 Bit RSA Integers in 8 Hours Using 20 Million Noisy Qubits. *Quantum*, 5:433, 2021. https://doi.org/10.22331/q-2021-04-15-433 doi:10.22331/q-2021-04-15-433.

[18] Jason Goertzen and Douglas Stebila. Post-quantum signatures in DNSSEC via request-based fragmentation. arXiv:2211.14196 [cs.CR], 2022. URL: https://arxiv.org/abs/2211.14196, https://doi.org/10.48550/arXiv.2211.14196 doi:10.48550/arXiv.2211.14196.

[19] Lov K. Grover. A Fast Quantum Mechanical Algorithm for Database Search. In *Proceedings of the 28th Annual ACM Symposium on Theory of Computing (STOC)*, pages 212–219. ACM, 1996. https://doi.org/10.1145/237814.237866 doi:10.1145/237814.237866.

[20] Hickory DNS Contributors. Hickory DNS: A rust-based DNS client, server, and resolver, 2024. URL: https://github.com/hickory-dns/hickory-dns.

[21] P. Hoffman and P. McManus. DNS Queries over HTTPS (DoH). RFC 8484, October 2018. URL: https://www.rfc-editor.org/rfc/rfc8484.

[22] Z. Hu, L. Zhu, J. Heidemann, A. Mankin, D. Wessels, and P. Hoffman. Specification for DNS over Transport Layer Security (TLS). RFC 7858, May 2016. URL: https://www.rfc-editor.org/rfc/rfc7858.

[23] IETF DNS Operations Working Group. Post-quantum cryptography for DNSSEC. IETF DNSOP Working Group documents, 2024. Work in progress; see https://datatracker.ietf.org/group/dnsop/documents/.

[24] B. Laurie, A. Langley, and E. Kasper. Certificate Transparency. RFC 6962, 2013. https://doi.org/10.17487/RFC6962 doi:10.17487/RFC6962.

[25] E. Lewis and A. Hoenes. DNS Zone Transfer Protocol (AXFR). RFC 5936, June 2010. URL: https://www.rfc-editor.org/rfc/rfc5936.

[26] Ralph C. Merkle. A Digital Signature Based on a Conventional Encryption Function. In *Advances in Cryptology – CRYPTO '87*, volume 293 of *Lecture Notes in Computer Science*, pages 369–378. Springer, 1988. URL: https://dl.acm.org/doi/10.5555/646752.704751.

[27] Giovane C. M. Moura, Moritz Müller, Marco Davids, Maarten Wullink, and Cristian Hesselman. Cache me if you can: Effects of DNS time-to-live. In *Proceedings of the 2019 Internet Measurement Conference (IMC)*, pages 101–115. ACM, 2019. URL: https://dl.acm.org/doi/10.1145/3355369.3355568, https://doi.org/10.1145/3355369.3355568 doi:10.1145/3355369.3355568.

[28] Moritz Müller, Jins de Jong, Martin van Hensbergen, et al. Retrofitting Post-Quantum Cryptography in Internet Protocols: A Case Study of DNSSEC. In *ACM SIGCOMM Computer Communication Review*, volume 50, pages 49–57, 2020. https://doi.org/10.1145/3431832.3431838 doi:10.1145/3431832.3431838.

[29] National Institute of Standards and Technology. SHA-3 Standard: Permutation-Based Hash and Extendable-Output Functions. Technical Report FIPS 202, NIST, 2015. URL: https://doi.org/10.6028/NIST.FIPS.202.

[30] National Institute of Standards and Technology. Status Report on the Third Round of the NIST Post-Quantum Cryptography Standardization Process. Technical Report NISTIR 8413, NIST, 2022. URL: https://doi.org/10.6028/NIST.IR.8413-upd1.

[31] National Institute of Standards and Technology. Module-Lattice-Based Digital Signature Standard (ML-DSA). Technical Report FIPS 204, NIST, 2024. URL: https://doi.org/10.6028/NIST.FIPS.204.

[32] National Institute of Standards and Technology. Post-Quantum Cryptography Project: Overview, Security Analyses, and FIPS PQ Standards. NIST CSRC project hub, 2024. Hub for the NIST PQC standardisation effort, selected algorithms, security analyses, and the finalised FIPS 203/204/205 standards. URL: https://csrc.nist.gov/projects/post-quantum-cryptography.

[33] National Institute of Standards and Technology. Stateless Hash-Based Digital Signature Standard (SLH-DSA). Technical Report FIPS 205, NIST, 2024. URL: https://doi.org/10.6028/NIST.FIPS.205.

[34] Benedikt Schmidt, Simon Meier, Cas Cremers, and David Basin. Automated analysis of diffie-hellman protocols and advanced security properties. In *25th IEEE Computer Security Foundations Symposium (CSF)*, pages 78–94, 2012. https://doi.org/10.1109/CSF.2012.25 doi:10.1109/CSF.2012.25.

[35] S. Sheth. A Strategy for Post-Quantum Cryptography in DNSSEC. Internet-Draft draft-sheth-pqc-dnssec-strategy-00, Internet Engineering Task Force, 2024. Work in progress. URL: https://www.ietf.org/archive/id/draft-sheth-pqc-dnssec-strategy-00.html.

[36] Peter W. Shor. Algorithms for Quantum Computation: Discrete Logarithms and Factoring. In *Proceedings of the 35th Annual Symposium on Foundations of Computer Science (FOCS)*, pages 124–134. IEEE, 1994. https://doi.org/10.1109/SFCS.1994.365700 doi:10.1109/SFCS.1994.365700.

[37] TLSNotary et al. zkTLS: Zero-Knowledge Proofs for TLS Sessions, 2024. See also: Reclaim Protocol, Opacity Network. URL: https://tlsnotary.org/.

[38] Paul Wouters and Ondřej Sury. Algorithm Implementation Requirements and Usage Guidance for DNSSEC, 2019. Internet Engineering Task Force. URL: https://www.rfc-editor.org/info/rfc8624, https://doi.org/10.17487/RFC8624 doi:10.17487/RFC8624.

[39] Mark Zhandry. How to Record Quantum Queries, and Applications to Quantum Indifferentiability. In *Advances in Cryptology – CRYPTO 2019*, volume 11693 of *Lecture Notes in Computer Science*, pages 239–268. Springer, 2019. URL: https://link.springer.com/chapter/10.1007/978-3-030-26951-$7_9, https://doi.org/10.1007/978-3-030-26951-7_9 doi:10.1007/978-3-030-26951-7_9$.

Appendix A
Per-algorithm phase breakdowns

Detailed per-phase wall-clock decompositions, measured on AWS `r8g.4xlarge` (Graviton4 / Neoverse-V2, DDR5-5600, 16 vCPU, `RAYON_NUM_THREADS=16`). Phases: trace fill, LDE expansion (blowup 32), streaming merge, STIR prove, local STIR verify. Every row is reproducible via the named `cargo run --release -p swarm-dns --example` command; the Ed25519 and ECDSA breakdowns are extracted from the `crossalg_three_signature_bench` log to share identical hardware state with the totals in §VI-C.

TABLE VIII
ECDSA-P256 PER-SIG PHASE BREAKDOWN
(`PROVE_ECDSA_RECORD_V1` / `CROSSALG`, RFC 6979 §A.2.5, K=256, CONS/ROW=155,774, n_0=8,192).

Phase	Wall time
Trace fill (real u_1, u_2)	548.71 ms
LDE expansion	88.49 s
Streaming merge	5.04 s
STIR prove	909.92 ms
Local STIR verify	1.20 ms
Total per ECDSA-P256 sig	**95.00 s (1.58 min)**

TABLE IX
RSA-2048 PER-SIG PHASE BREAKDOWN
(`PROVE_RSA2048_RECORD_V1`, e=65,537 EXP CHAIN, 1.16×10^7 CONSTRAINT EVALUATIONS, n_{trace}=32, n_0=1,024).

Phase	Wall time
Trace fill (real s, n, em)	1.45 ms
LDE expansion	613.56 ms
Streaming merge	50.90 ms
STIR prove	5.41 ms
Local STIR verify	1.01 ms
Total per RSA-2048 sig	**0.68 s**

TABLE X
ED25519 PER-SIG PHASE BREAKDOWN (`CROSSALG`, RFC 8032 §5.1.7 COFACTORED, K=256, CONS/ROW=70,606, n_0=32,768).

Phase	Wall time
Trace fill	268.35 ms
LDE expansion	100.05 s
Streaming merge	3.61 s
STIR prove	1.08 s
Local STIR verify	1.21 ms
Total per Ed25519 sig	**105.01 s (1.75 min)**

TABLE XI
EDGE-CONSUMER ONCE-PER-EPOCH PHASE BREAKDOWN
(`EDGE_CONSUMER_FULL_DEMO`, N=10 RSA RECORDS). REAL FIPS-204 ML-DSA-65 (NO STAND-IN APPROXIMATION) AND REAL SHA3-512 MERKLE PATH VERIFY.

Phase	Wall time
ML-DSA-65 verify	81.25 μs
STARK verify (STIR-based)	484.21 μs
Once-per-epoch total	**567.71 μs**
Per-query Merkle path (depth 4)	791 ns

TABLE XII
AIRGAPPED RESOLVER BENCH (`AIRGAPPED_RESOLVER_DEMO`, N=50, M=1,000 QUERIES; BUNDLE ON DISK 118 KIB; NO NETWORK ACCESS DURING THE QUERY PHASE).

Phase	Wall time
Bundle parse from disk	62 μs
ML-DSA-65 verify	86 μs
STARK verify (STIR-based)	672 μs
Once-per-epoch boot	**868 μs**
Per-query (avg over 1,000)	1.156 μs
1,000 queries cumulative	1.16 ms

Appendix B
Tamarin Theory for the Swarm Protocol

This appendix reproduces the Tamarin theory referenced in §V-E. It auto-proves all ten lemmas in < 1 s on tamarin-prover 1.10.0 + maude 2.7.1; the full source is included verbatim below for review-time re-verification.

A. Modelling Decisions

- TLS confidentiality is NOT modelled: every wire message is sent `Out(...)` and read `In(...)` in the clear. Authentication is supplied by ML-DSA signatures (Tamarin's `signing` builtin). This matches the threat model where the adversary controls the network anyway.
- Hash digests are flattened to `h(<...>)`—the free function symbol `h/1` is collision-resistant by construction (injective on tuples), which is what the symbolic proof needs.
- Worker / primary / witness key compromise is exposed as adversary rules `Compromise_Worker` and `Compromise_Witness`.
- A `[sources, reuse]` source lemma feeds the harder downstream lemmas (`shard_inclusion`, `witness_quorum_`M).

B. Setup and Adversary Rules

```
theory STIR_DNS_Swarm
begin

builtins: signing, hashing

/* Long-term keys */
rule Worker_KeyGen:
    [ Fr(~sk) ]
  --[ WKey($W, ~sk) ]->
    [ !Wsk($W, ~sk),
```

```
      !Wpk($W, pk(~sk)),
      Out(pk(~sk)) ]

rule Ctrl_KeyGen:
    [ Fr(~sk) ]
  --[ CKey($C, ~sk) ]->
    [ !Csk($C, ~sk),
      !Cpk($C, pk(~sk)),
      Out(pk(~sk)) ]

rule Witness_KeyGen:
    [ Fr(~sk) ]
  --[ VKey($V, ~sk) ]->
    [ !Vsk($V, ~sk),
      !Vpk($V, pk(~sk)),
      Out(pk(~sk)) ]

rule Allowlist:
    [ !Csk($C, skC), !Wpk($W, pkW) ]
  --[ Allowlisted($C, $W) ]->
    [ !Allow($C, $W) ]

/* Adversary key compromise */
rule Compromise_Worker:
    [ !Wsk($W, skW) ]
  --[ CompromisedW($W) ]->
    [ Out(skW) ]

rule Compromise_Witness:
    [ !Vsk($V, skV) ]
  --[ CompromisedV($V) ]->
    [ Out(skV) ]
```

C. Protocol Rules

```
/* Phase 2: assignment */
rule Assign:
    [ !Csk($C, skC), !Wpk($W, pkW),
      !Allow($C, $W),
      Fr(~jobId), Fr(~nonce) ]
  --[ Assigned($C, $W, ~jobId, $sid,
               ~nonce, $records) ]->
    [ Out(<'assign', $C, $W, ~jobId,
          $sid, ~nonce, $records>),
      St_Assign($C, $W, ~jobId, $sid,
                ~nonce, $records) ]

/* Phase 3: worker proves and signs attestation */
rule WorkerProves:
    let
      fs     = h(<'fs', $C, jobId, sid, nonce>)
      pi     = h(<'pi', records, fs>)
      digest = h(<'attest', sid, pi>)
      sigW   = sign(digest, ~skW)
    in
    [ !Wsk($W, ~skW),
      In(<'assign', $C, $W, jobId, sid,
          nonce, records>) ]
  --[ Proved($W, $C, jobId, sid,
             records, pi, sigW),
      ProvedFor($W, $C, jobId, sid,
                nonce, records, pi) ]->
    [ Out(<'attest', $W, $C, jobId, sid,
          pi, sigW>) ]

/* Phase 4: ctrl accepts, mints receipt */
rule CtrlAccept:
    let
      digest = h(<'attest', sid, pi>)
      sigC   = sign(<'receipt', sid, pi>, ~skC)
    in
    [ !Csk($C, ~skC), !Wpk($W, pkW),
      !Allow($C, $W), Fr(~ts),
      In(<'attest', $W, $C, jobId, sid,
          pi, sigW>) ]
  --[ Eq(verify(sigW, digest, pkW), true),
      AcceptedShard($C, $W, jobId, sid,
                    pi, sigW),
      MintedReceipt($C, $W, jobId, sid,
                    pi, sigC, ~ts) ]->
    [ Out(<'receipt', $C, $W, jobId,
          sid, pi, sigC, ~ts>) ]

/* Witness archival: each witness independently
 * verifies and archives its own receipt */
rule WitnessArchive:
    let
      sigV = sign(<'wreceipt', jobId, sid,
                   pi>, ~skV)
    in
    [ !Vsk($V, ~skV), Fr(~tsV),
      In(<'attest', $W, $C, jobId, sid,
          pi, sigW>) ]
  --[ WitnessReceipt($V, $W, jobId, sid,
                     pi, sigV, ~tsV) ]->
    [ Out(<'wreceipt', $V, $W, jobId,
          sid, pi, sigV, ~tsV>) ]

restriction Equality:
    "All x y #i. Eq(x,y) @ i ==> x = y"
```

D. Verified Lemmas

```
/* Source lemma: every Proved fact originates
 * from an honest worker holding skW. */
lemma proved_origin [sources, reuse]:
  "All W C j s records pi sig #i.
       Proved(W, C, j, s, records, pi, sig) @ i
   ==> ( Ex skW #k.
            WKey(W, skW) @ k & k < i )"

/* Sanity / liveness */
lemma protocol_executable: exists-trace
  "Ex C W j s pi sig #i.
        AcceptedShard(C, W, j, s, pi, sig) @ i"

/* E1: allowlist enforcement */
lemma allowlist_enforced [reuse]:
  "All C W j s pi sig #i.
       AcceptedShard(C, W, j, s, pi, sig) @ i
   ==> Ex #a. Allowlisted(C, W) @ a & a < i"

/* Property 1: authority binding */
lemma authority_binding [reuse]:
  "All C W j s pi sigC ts #i.
       MintedReceipt(C, W, j, s, pi,
                     sigC, ts) @ i
   ==> Ex sk #k. CKey(C, sk) @ k & k < i"

/* Property 2: shard inclusion --
 * accepted shards are either backed by an honest
 * worker's proof, OR the worker key was
 * adversarially compromised. */
lemma shard_inclusion [reuse]:
  "All C W j s pi sig #i.
       AcceptedShard(C, W, j, s, pi, sig) @ i
   ==> ( Ex records #p.
            Proved(W, C, j, s, records,
                   pi, sig) @ p & p < i )
     | ( Ex #c. CompromisedW(W) @ c & c < i )"

/* Property 3: replay freshness --
 * the same pi cannot belong to two distinct
 * (jobId, sid) shards. */
lemma replay_freshness [reuse]:
  "All C1 W1 j1 s1 pi sig1
       C2 W2 j2 s2 sig2 #t1 #t2.
       AcceptedShard(C1, W1, j1, s1,
                     pi, sig1) @ t1
     & AcceptedShard(C2, W2, j2, s2,
```

```
                    pi, sig2) @ t2
   ==> j1 = j2 & s1 = s2"

/* Property 4: deterministic proving */
lemma deterministic_proving:
  "All W1 W2 C j s nonce records
       pi1 pi2 #t1 #t2.
       ProvedFor(W1, C, j, s, nonce,
                 records, pi1) @ t1
     & ProvedFor(W2, C, j, s, nonce,
                 records, pi2) @ t2
   ==> pi1 = pi2"

/* Receipt provenance */
lemma receipt_provenance:
  "All C W j s pi sigC ts #i.
       MintedReceipt(C, W, j, s, pi,
                     sigC, ts) @ i
   ==> Ex sigW.
         AcceptedShard(C, W, j, s,
                       pi, sigW) @ i"

/* Property 5 (M=2): witness quorum */
lemma witness_quorum_2 [reuse]:
  "All V1 V2 W1 W2 j s pi
       s1 s2 t1 t2 #i1 #i2.
       WitnessReceipt(V1, W1, j, s,
                      pi, s1, t1) @ i1
     & WitnessReceipt(V2, W2, j, s,
                      pi, s2, t2) @ i2
     & not(V1 = V2)
   ==> ( Ex sk1 #k1.
             VKey(V1, sk1) @ k1 & k1 < i1 )
     & ( Ex sk2 #k2.
             VKey(V2, sk2) @ k2 & k2 < i2 )"

/* Property 5 (M=3) */
lemma witness_quorum_3:
  "All V1 V2 V3 W1 W2 W3 j s pi
       s1 s2 s3 t1 t2 t3 #i1 #i2 #i3.
       WitnessReceipt(V1, W1, j, s,
                      pi, s1, t1) @ i1
     & WitnessReceipt(V2, W2, j, s,
                      pi, s2, t2) @ i2
     & WitnessReceipt(V3, W3, j, s,
                      pi, s3, t3) @ i3
     & not(V1 = V2)
     & not(V1 = V3) & not(V2 = V3)
   ==> ( Ex sk1 #k1.
             VKey(V1, sk1) @ k1 & k1 < i1 )
     & ( Ex sk2 #k2.
             VKey(V2, sk2) @ k2 & k2 < i2 )
     & ( Ex sk3 #k3.
             VKey(V3, sk3) @ k3 & k3 < i3 )"

end
```

E. Reproducing the Verification

The verification is deterministic and terminates in well under a second on commodity hardware:

```
$ tamarin-prover --prove security/swarm.spthy
```

Each lemma's status is reported as `verified`, with step counts ranging from 2 to 13. The `[sources, reuse]` source lemma plus three `[reuse]` chained intermediate lemmas (`allowlist_enforced`, `authority_binding`, `shard_inclusion`) provide heuristic guidance to the backend so no oracle script is required. Total wall-clock on tamarin-prover 1.10.0 + maude 2.7.1 is < 1 s.